MATH6313 Lecture Notes

Caelen Gelineau

October 20, 2026

Contents

1 Lecture 1 - 25/08/26 **3**

2 Lecture 2 - 27/08/26 **3**

3 Lecture 9 - 22/09/26 **3**
3.1 LU Factorization . . . 3
3.2 Summary of LU Factorization method . . . 4
3.3 Solving $Ax = b$ . . . 4
3.4 Additional pivoting matrix P . . . 5
3.5 Gaussian Elimination vs LU factorization efficiency . . . 5
3.5.1 Operation count for back substitution (LU method) . . . 5
3.5.2 Operation count for Gaussian elimination . . . 6
3.6 $PA = LU$ factorization, or Matrix form of Gaussian elimination w/ partial pivoting 7
3.6.1 Summary . . . 8

4 Lecture 10 - 24/09/26 **9**
4.1 Iterative methods for $Ax = b$ . . . 9
4.1.1 Alternate derivation . . . 9

5 Lecture 11 - 29/09/26 **9**
5.1 Iterative Methods for Ax = b . . . 9
5.2 Jacobi Method . . . 10
5.3 Gauss-Seidel method . . . 11

6 Lecture 12 - 01/10/26 **13**
6.1 Line search method . . . 13
6.2 Steepest descent . . . 13
6.3 Interpolation . . . 14
6.4 Lagrange Interpolation . . . 14

7 Lecture 13 - 08/10/26 **15**
7.1 Chebyshev nodes and polynomials . . . 15

8 Lecture 14 - 13/10/26 **17**
8.1 Cubic spline interpolation . . . 18

9 Lecture 15 - 15/10/26 **19**
9.1 Least squares method 19
9.2 Conditioning of least-squares 20
9.3 QR factorization and Gram-Schmidt orthogonalization 20

10 Lecture 16 - 20/10/26 **21**
10.1 Review of interpolation methods 21
10.2 Gram-Schmidt orthogonalization for least-squares method 22
10.3 Classical Gram-Schmidt algorithm 23
10.4 Special case with small numbers 23
10.5 Modified Gram-Schmidt algorithm 24
10.6 Reduced QR 24

1 Lecture 1 - 25/08/26

Calculus review.

2 Lecture 2 - 27/08/26

Theorem 2.1 (Taylor's theorem)**.** *Suppose $f \in C^n[a,b]$ that f^{n+1} exists on $[a,b]$ and $x_0 \in [a,b]$. $\forall x \in [a,b], \exists \xi(x)$ between x_0 and x s.t.*

$$f(x) = P_n(x) + R_n(x)$$

where

$$P_n(x) = f(x_0) + f'(x_0)(x - x_0) + \frac{1}{2!}f''(x_0)(x - x_0)^2 + \cdots + \frac{1}{n!}f^{(n)}(x_0)(x - x_0)^n$$

$$R_n(x) = \frac{f^{(n+1)}(\xi(x))}{(n+1)!}(x - x_0)^{(n+1)}$$

3 Lecture 9 - 22/09/26

3.1 LU Factorization

Definition 3.1. Lower triangular matrix

$$\begin{pmatrix} x & 0 \\ x & x \end{pmatrix}$$

Definition 3.2. Upper triangular matrix

$$\begin{pmatrix} x & x \\ 0 & x \end{pmatrix}$$

Want to write the matrix as

$$A = LU$$

Example 3.3.

$$\begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix} = \begin{pmatrix} 2 & -1 \\ 0 & \frac{3}{2} \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ -\frac{1}{2} & 1 \end{pmatrix}$$

Example 3.4. Define

$$A = \begin{pmatrix} 1 & 2 & 1 \\ 3 & 8 & 1 \\ 0 & 4 & 1 \end{pmatrix}$$

$$L_{21}(-3) = \begin{pmatrix} 1 & 0 & 0 \\ -3 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

$$L_{32}(-2) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -2 & 1 \end{pmatrix}$$

$$L_{21}(-3)A = \begin{pmatrix} 1 & 0 & 0 \\ -3 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 2 & 1 \\ 3 & 8 & 1 \\ 0 & 4 & 1 \end{pmatrix}$$

Repeat, which results in

$$L_{32}(-2)L_{21}(-3)A = \cdots = U$$

Now we get

$$A = LU$$

$$L^{-1} = L_{32}(-2)L_{21}(-3)$$

$$L = L_{21}^{-1}(-3)L_{32}^{-1}(-2) = \begin{pmatrix} 1 & 0 & 0 \\ 3 & 1 & 0 \\ 0 & 2 & 1 \end{pmatrix}$$

3.2 Summary of LU Factorization method

Facts:

1. $L_{ij}(-c)$ denotes the lower triangular matrix whose only non-zero entries are 1's on the diagonal, and $-c$ is in the (i, j) position. Then $A \to L_{ij}(-c)A$ represents the RREF operation "row - row $\times c$".
2. $[L_{ij}(c)]^{-1} = L_{ij}(-c)$
3. From the previous example, $L_{21}(3)L_{32}(2) = \begin{pmatrix} 1 & 0 & 0 \\ 3 & 1 & 0 \\ 0 & 2 & 1 \end{pmatrix}$ which gives us the full lower triangular matrix desired for LU factorization, which you can then use to find $U = L^{-1}A$.

3.3 Solving $Ax = b$

Given $LU = A$, you can solve $Ax = b$ by

1. $Lc = b$ (top to bottom)
2. $Ux = c$ (bottom to top)

Example 3.5.

$$LUx = Lc = b$$

$$\begin{pmatrix} 1 & 1 \\ 3 & -4 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 3 & 1 \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 0 & -7 \end{pmatrix}$$

Using "$Lc = b$",

$$\begin{pmatrix} 1 & 0 \\ 3 & 1 \end{pmatrix} \begin{pmatrix} c_1 \\ c_2 \end{pmatrix} = \begin{pmatrix} 3 \\ 2 \end{pmatrix}$$

solving from top equation to bottom equation yields

$$\Rightarrow \begin{cases} c_1 = 3 \\ c_2 = -7 \end{cases}$$

Now using "$Ux = c$"

$$\begin{pmatrix} 1 & 1 \\ 0 & -7 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 3 \\ -7 \end{pmatrix}$$

solve from bottom equation to top equation yields

$$\Rightarrow \begin{cases} c_1 = 2 \\ c_2 = 1 \end{cases}$$

3.4 Additional pivoting matrix P

If you have $PA = LU$, and you can solve $Ax = b$ by the following:

$$PAx = Pb$$

and thus $PA = LU$. Thus, you solve

$$\begin{cases} Lc = Pb \\ Ux = c \end{cases}$$

Example 3.6.

$$\begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix} \begin{pmatrix} 2 & 1 & 5 \\ 4 & 4 & -4 \\ 1 & 3 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ \frac{1}{4} & 1 & 0 \\ \frac{1}{2} & -\frac{1}{2} & 1 \end{pmatrix} \begin{pmatrix} 4 & 4 & -4 \\ 0 & 2 & 2 \\ 0 & 0 & 8 \end{pmatrix}$$

Exercise to the reader.

3.5 Gaussian Elimination vs LU factorization efficiency

The computational resource for Gaussian is $\mathcal{O}(\frac{2}{3}n^3k)$, while the computational resource for LU factorization is $\mathcal{O}(2n^2k)$, where k is the dimension of the matrix.

3.5.1 Operation count for back substitution (LU method)

Suppose you have an $n \times n$ upper triangular matrix.

$$\begin{pmatrix} a_{11} & a_{12} \dots a_{1n} & b_1 \\ 0 & a_{22} \dots a_{2n} & b_2 \\ 0 & 0 \dots a_{nn} & b_n \end{pmatrix}$$

Then we solve

$$x_1 = \frac{b_1 - a_{12}x_2 - \cdots - a_{1n}x_n}{a_{11}}$$

$$\vdots$$

$$x_n = \frac{b_n}{a_{nn}}$$

So we have for computing x_1, 1 division operation, $n-1$ multiplications, and $n-1$ additions so we have $2n-1$ total operations. then for x_2, you have $2n-3$ operations, all the way until $1+3+5+\cdots+(2n-1) = n^2$ operations for x_n. So we have $\mathcal{O}(n^2)$.

3.5.2 Operation count for Gaussian elimination

Suppose you have an $n \times n$ matrix.

$$\begin{pmatrix} a_{11} & a_{12} \dots a_{1n} & b_1 \\ a_{21} & a_{22} \dots a_{2n} & b_2 \\ a_{n1} & a_{n2} \dots a_{nn} & b_n \end{pmatrix}$$

The function we want to write is:

```
for j = 1: n-1
    eliminate column j ⟶ zero below main diagonal
```

which is the same as

```
for j = 1: n-1
    for i = j+1:n
        eliminate a(i,j)
    end
end
```

explicitly, the algorithm is

```
for j = 1: n-1
    for i = j+1:n
        mult = a(i,j) / a(j,j)
        for k = j+1 : n
          a(i,k) = a(i,k)-mult * a(j,k)
        end
    end
end
```

To "eliminate a_{21}",

$$\begin{pmatrix} a_{11} & a_{12} & \dots & a_{1n} & b_1 \\ 0 & a_{22} - \frac{a_{21}}{a_{11}} a_{12} & \dots & a_{2n} - \frac{a_{21}}{a_{11}} a_{2n} & b_2 - \frac{a_{21}}{a_{11}} b_1 \\ a_{n1} & a_{n2} & \dots & a_{nn} & b_n \end{pmatrix}$$

which is 1 division, n multiplications, and n additions and subtractions. Thus we have a total of $2n+1$ operations. We have the following table for the number of operations for each matrix entry. The number of operations for each column decreases by 1 as you move left to right since we have one less entry to worry about.

$$\begin{bmatrix} 0 & & & \\ 2n+1 & 0 & & \\ 2n+1 & 2(n-1)+1 & 0 & \\ \vdots & \vdots & \vdots & \\ 2n+1 & 2(n-1)+1 & 2(n-2)+1 & \\ 2n+1 & 2(n-1)+1 & 2(n-2)+1 & 0 \end{bmatrix}$$

Then the sum of total operations is

$$\sum_{j=1}^{n-1} \sum_{i=1}^{j} [2(j+1)+1] = \sum_{j=1}^{n-1} [2j(j+1)+j] = \mathcal{O}(\frac{2}{3}n^3)$$

3.6 $PA = LU$ factorization, or Matrix form of Gaussian elimination w/ partial pivoting

Example 3.7. Note, the fractions arent actually there, just to track for L.

$$A = \begin{pmatrix} 2 & 1 & 5 \\ 4 & 4 & -4 \\ 1 & 3 & 1 \end{pmatrix} \longrightarrow \begin{pmatrix} 4 & 4 & -4 \\ 2 & 1 & 5 \\ 1 & 3 & 1 \end{pmatrix} \longrightarrow_{r_2 - \frac{1}{2} r_1, \quad r_3 - \frac{1}{4} r_1} \begin{pmatrix} 4 & 4 & -4 \\ \frac{1}{2} & -1 & 7 \\ \frac{1}{4} & 2 & 2 \end{pmatrix} \longrightarrow \begin{pmatrix} 4 & 4 & -4 \\ \frac{1}{4} & 2 & 2 \\ \frac{1}{2} & -1 & 7 \end{pmatrix}$$

$$\longrightarrow_{r_3 - (-\frac{1}{2} r_2)} \begin{pmatrix} 4 & 4 & -4 \\ \frac{1}{4} & 2 & 2 \\ \frac{1}{2} & -\frac{1}{2} & 8 \end{pmatrix}$$

or equivalently

$$P_1 A, \quad P_1 = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

$$M_1P_1A, \quad M_1 = \begin{pmatrix} 1 & 0 & 0 \\ -\frac{1}{2} & 1 & 0 \\ -\frac{1}{4} & 0 & 1 \end{pmatrix}$$

$$P_2M_1P_1A, \quad P_2 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix}$$

$$M_2P_2M_1P_1A = U, \quad M_2 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & \frac{1}{2} & 1 \end{pmatrix}$$

thus combining we get

$$U = \begin{pmatrix} 4 & 4 & -4 \\ 0 & 2 & 2 \\ 0 & 0 & 8 \end{pmatrix}$$

$$L = \begin{pmatrix} 1 & 0 & 0 \\ \frac{1}{4} & 1 & 0 \\ \frac{1}{2} & -\frac{1}{2} & 1 \end{pmatrix}$$

$$P = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix}$$

where P tracks the row swapping operations by multiplying the P_i matrices.
We can equivalently write

$$M_2P_2M_1P_1A = U$$

$$M_2P_2M_1P_2^{-1}P_2P_1A = U$$

$$M_2P_2M_1P_2^{-1}PA = U$$

$$L = (P_2M_1P_2^{-1})M_2^{-1}$$

Fact: $P_2M_1P_2^{-1}$ has same structure as M_1.

3.6.1 Summary

Definition 3.8. A permuatation matrix P is an $n \times n$ matrix consisting of all zeroes except for a single 1 in every row and column.

In general you have

$$M_{m-1}P_{m-1}\cdots M_2P_2M_1P_1A = U \iff M'_{m-1}M'_{m-2}\cdots M'_1P_{m-1}P_{m-2}\cdots P_1A = U$$

where $M'_{m-1} = M_{m-1}$, $M'_{m-2} = P_{m-1}M_{m-2}P_{m-1}^{-1}, \ldots, M'_1 = P_{m-1}P_{m-2}\cdots P_2M_1P_2^{-1}\cdots P_{m-1}^{-1}$.

4 Lecture 10 - 24/09/26

4.1 Iterative methods for $Ax = b$

Let x be the exact solution and the initial guess is x_0. Define $e := x - x_0$.

Definition 4.1. The residual is defined as $r := b - Ax_0$.

Notice that

$$Ae = A(x - x_0) = Ax - Ax_0 = b - Ax_0 = r$$

Thus, the error can be computed if the residual is known:

$$e = A^{-1}(b - Ax_0)$$

$$x = x_0 + A^{-1}(b - Ax_0)$$

however, we do not want to compute A^{-1}, so we will find something to replace it, giving the following iterative method:

$$x_{k+1} = x_k + B(b - Ax_k)$$

This turns out to be fixed point iteration:

$$x_{k+1} = G(x_k), \quad G(x) = x + B(b - Ax)$$

Write

$$A = L + D + U$$

where D is a strictly diagonal matrix and L, U as expected. We can then define $B := D^{-1}$ or $B := (L + D)^{-1}$.

4.1.1 Alternate derivation

$$Ax = b$$

$$(L + D + U)x = b$$

$$Lx + Dx + Ux = b$$

$$Lx_k + Dx_{k+1} + Ux_k = b$$

$$x_{k+1} = D^{-1}(b - Lx_k - Ux_k)$$

5 Lecture 11 - 29/09/26

5.1 Iterative Methods for Ax = b

Idea: say the exact solution is x and the initial guess is x_0. The error is $e = x - x_0$. The residual is $r = b - Ax_0$. We can relate the residual to the error by

$$Ae = A(x - x_0) = Ax - Ax_0 = b - Ax_0 = r$$

$$e = A^{-1}(b - Ax_0)$$

$$x = x_0 + A^{-1}(b - Ax_0)$$

The iterative method is then

$$x_{k+1} = x_k + B(b - Ax_k)$$

where B replaces the inverse of A^{-1}. Examples for B are

1. $B = D^{-1}$
2. $B = (D + L)^{-1}$

where $A = L + D + U$ where L is a lower matrix, D is a diagonal matrix, and U is a strictly upper matrix.

5.2 Jacobi Method

We take $B = D^{-1}$.

$$\begin{aligned} x_{k+1} &= x_k + D^{-1}(b - Ax_k) \\ &= x_k + D^{-1}(b - (L + D + U)x_k) \\ &= D^{-1}(b - (L + U)x_k) \end{aligned}$$

Which also can be written as

$$Dx_{k+1} = b - (L + U)x_k$$

or

$$Dx_{k+1} + (L + U)x_k = b$$

Example 5.1.

$$\begin{pmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 2 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}$$

$$L = \begin{pmatrix} 0 & 0 & 0 \\ -1 & 0 & 0 \\ 0 & -1 & 0 \end{pmatrix}, \quad D = \begin{pmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 2 \end{pmatrix}, \quad U = \begin{pmatrix} 0 & -1 & 0 \\ 0 & 0 & -1 \\ 0 & 0 & 0 \end{pmatrix}$$

Then we have the system

$$\begin{cases} 2x_{k+1} - y_k + 0 = 0 \\ -x_k + 2y_{k+1} - z_k = 1 \\ 0 - y_k + 2z_{k+1} = 0 \end{cases}$$

which solves to

$$\begin{cases} x_{k+1} = \frac{y_k}{2} \\ y_{k+1} = \frac{1}{2}(1 + x_k + z_k) \\ z_{k+1} = \frac{y_k}{2} \end{cases}$$

Definition 5.2. The $n \times n$ matrix $A = (a_{ij})$ is strictly diagonally dominant if for any $1 \leq i \geq m$, we have

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}|$$

Example 5.3.

$$\begin{pmatrix} 3 & 1 & -1 \\ 2 & -5 & 2 \\ 1 & 6 & 8 \end{pmatrix}$$

is strictly diagonally dominant, while

$$\begin{pmatrix} 3 & 2 & 6 \\ 1 & 8 & 1 \\ 3 & 2 & -2 \end{pmatrix}$$

is NOT strictly diagonally dominant.

Theorem 5.4. *If A is strictly diagonally dominant, then A is non-singular ($Ax = b$ has a solution). Additionally, for every b and initial guess x_0, the Jacobi method converges to the unique solution. This also applies to the Gauss-Seidel method.*

5.3 Gauss-Seidel method

For this method, we have $B = (L + D)^{-1}$.

$$\begin{aligned} x_{k+1} &= x_k + (L + D)^{-1}(b - Ax - k) \\ &= x_k + (L + D)^{-1}(b - (L + D + U)x_k) \\ &= (L + D)^{-1}(b - Ux_k) \end{aligned}$$

which can be written as

$$(L + D)x_{k+1} + Ux_k = b$$

Example 5.5.

$$\begin{pmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 2 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}$$

$$L = \begin{pmatrix} 0 & 0 & 0 \\ -1 & 0 & 0 \\ 0 & -1 & 0 \end{pmatrix}, \quad D = \begin{pmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 2 \end{pmatrix}, \quad U = \begin{pmatrix} 0 & -1 & 0 \\ 0 & 0 & -1 \\ 0 & 0 & 0 \end{pmatrix}$$

Then we have the system

$$\begin{cases} 2x_{k+1} - y_k + 0 = 0 \\ -x_{k+1} + 2y_{k+1} - z_k = 1 \\ 0 - y_{k+1} + 2z_{k+1} = 0 \end{cases}$$

$$L + U = \begin{pmatrix} 2 & 0 & 0 \\ -1 & 2 & 0 \\ 0 & -1 & 2 \end{pmatrix}$$

which solves to

$$\begin{cases} x_{k+1} = \frac{y_k}{2} \\ y_{k+1} = \frac{(1 + x_{k+1} + z_k)}{2} \\ z_{k+1} = \frac{y_{k+1}}{2} \end{cases}$$

The $(L + D)^{-1}$ is exactly forward substitution method.

Lemma 5.6. *If A is strictly diagonally dominant, then* $\det(A) \neq 0$*, i.e. A is non-singular.*

Proof. If $\det(A) = 0$, then A has zero eigenvalues, i.e. $Ax = 0$ for some non-zero x. Then $\sum_{j=1}^{n} a_{ij}x_j = 0$ for all $i = 1, 2, \ldots, n$. Take the zero eigenvalue and set it equal to a_{ii}.

$$|a_{ii} \cdot x_i| = |a_{ii}||x_i| = \sum_{j \neq i} |a_{ij}x_j| \leq \max_{j \neq i} |x_j| \sum_{j \neq i} |a_{ij}| < \max_{j \neq i} |x_j| \cdot |a_{ii}|$$

$$\implies |x_i| < \max_{i \neq j} |x_j|, \quad \forall i = 1, 2, \cdots, n$$

which is a contradiction with s.d.d. □

Lemma 5.7. *If the $n \times n$ matrix A has spectral radius $\rho(A) < 1$ (all eigenvalues bounded above by 1), where*

$$\rho(A) := \max_{1 \leq i \leq n} \{|\lambda_i| : \lambda \text{ eigenvalue of } A\}$$

then for any b and x_0, $x_{k+1} = Ax_k + b$ converges. In particular, $\lim_{k \to \infty} x_k = x_*$ with $x_* = Ax_* + b$ and is unique.

Proof. Idea:

1. Uniqueness. Let $x_{*_1} = x_{*_2}$ such that $Ax_{*_i} = Ax_{*_i} + b$. Let $z_* = x_{*_1} - x_{*_2}$. Then $z_* = Az_* \implies 1$ is an eigenvalue of A, but $\rho(A) < 1$, so this is a contradiction.
2. Consider the geometric series $x_* = b + Ab + A^2b + \cdots$.

   $$x_* = Ax_* + b \iff (I - A)x_* = b$$

   this Neumann series converges due to $\rho(A) < 1$, thus a solution exists. Now, $e_k = x_k - x_*$.

   $$Ae_k = A(x_k - x_*) = Ax_k + b - (Ax_* + b) = x_{k+1} - x_* = e_{k+1}$$

   then

   $$\|e_{k+1}\| \leq \|Ae_k\| \leq \|A\|\|e_k\| \to 0$$

□

Proof of Theorem 3.4.

$$\begin{aligned} x_{k+1} &= x_k + D^{-1}(b - Ax_k) \\ &= (I - D^{-1}A)x_k + D^{-1}b \end{aligned}$$

By Lemma 3.7, it suffices to show that $\rho(I - D^{-1}A) < 1$.

$$(I - D^{-1}A)x = \lambda x$$

$$(D - A)x = \lambda Dx$$

$$(L + U)x = -\lambda Dx$$

$$|\lambda a_{ii}x_i| = |\sum_{i \neq j} a_{ij}x_j| \leq \max_{j \neq i} |x_j| \sum_{j \neq i} |a_{ij}| < \max_{j \neq i} |x_j| \cdot |a_{ii}|$$

Let $|x_k| = \max_{i=1,2,\ldots,n} |x_i| \neq 0$. Then

$$|\lambda a_{kk}x_k| \leq \max_{j \neq k} |x_j||a_{kk}| \leq |x_k||a_{kk}| \implies |\lambda| < 1 \implies \rho(I - D^{-1}A) < 1.$$

□

6 Lecture 12 - 01/10/26

Remark 6.1.

$$A = \begin{pmatrix} 1 & 4 \\ \frac{1}{8} & 1 \end{pmatrix}$$

is not strictly diagonally dominant, but $\rho(I - D^{-1}A) < 1 \implies$ convergence. So s.d.d. is a sufficient but not necessary condition.

6.1 Line search method

We will only deal with symmetric positive definite matrices.

$$A^T = A, \quad \forall x \neq 0,\ x^T A x > 0$$

Example 6.2.

$$\begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix}$$

is symmetric positive definite.

General method for line search method:

```
initial guess x_0
for k = 0, 1, ...
  choose search direction d_k
  choose step size alpha_k
  set x_(k+1) = x_k + \alpha_k * d_k
end
```

6.2 Steepest descent

We choose

$$d_k = r_k = b - Ax_k$$

$$\alpha_k = \frac{d_k^T r_k}{d_k^T A d_k}$$

Consider

$$f(x) = \frac{1}{2} x^T A x - b^T x$$

We want to minimize $f(x)$, so compute the gradient:

$$\nabla f(x) = Ax - b = -r$$

$$a_k = \min_{\alpha} \{f(x_k + \alpha d_k)\}$$

So to minimize,

$$\frac{\partial f}{\partial \alpha} = \nabla f \cdot d_k = [A(x_k + \alpha d_k) - b] \cdot d_k = (-r_k + \alpha A d_k) \cdot d_k = 0$$

$$(-r_k + \alpha A d_k) \cdot d_k = 0 \implies \alpha_k = \frac{d_k^T r_k}{d_k^T A d_k} = \frac{r_k^T r_k}{r_k^T A r_k}$$

Remark 6.3. We can use a better search direction, the conjugate gradient method.

6.3 Interpolation

Given a set of points, how can we find a polynomial that interpolates those points?

$$p(x_i) = y_i, \quad (x_i, y_i) \text{ are the given data points}$$

Theorem 6.4. *Given data points $(x_1, y_1), \ldots, (x_n, y_n)$, there exists a unique polynomial $p(x)$ of degree $n-1$ or less.*

$$p(x_i) = y_i, \quad i = 1, 2, \ldots, n$$

Goal: compute this polynomial $p(x)$.

Example 6.5. $(0,1), (2,2)$.

$$p(x) = a_0 + a_1 x$$

$$\begin{cases} a_0 + 0 * a_1 = 1 \\ a_0 + 2a_1 = 2 \end{cases} \implies \begin{cases} a_0 = 1 \\ a_1 = \frac{1}{2} \end{cases}$$

6.4 Lagrange Interpolation

Example 6.6. $(0,1), (2,2), (3,4)$. Define the following polynomials:

$$L_1(x) = \frac{(x - x_2)(x - x_3)}{(x_1 - x_2)(x_1 - x_3)}$$

$$L_2(x) = \frac{(x - x_1)(x - x_3)}{(x_2 - x_1)(x_2 - x_3)}$$

$$L_3(x) = \frac{(x - x_1)(x - x_2)}{(x_3 - x_1)(x_3 - x_2)}$$

Then we can compute

$$L_1(x_1) = 1, \ L_1(x_2) = 0, \ L_1(x_3) = 0$$
$$L_2(x_1) = 0, \ L_2(x_2) = 1, \ L_3(x_3) = 0$$
$$L_3(x_1) = 1, \ L_3(x_2) = 0, \ L_3(x_3) = 1$$

Then we can compute the polynomial as

$$p(x) = y_1 L_1(x) + y_2 L_2(x) + y_3 L_3(x)$$

Definition 6.7. Given $(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)$, define

$$L_1(x) = \frac{(x - x_2)(x - x_3) \cdots (x - x_n)}{(x_1 - x_2)(x_1 - x_3) \cdots (x_1 - x_n)}$$

$$L_k(x) = \frac{(x - x_1) \cdots (x - x_{k-1})(x - x_{k+1}) \cdots (x - x_n)}{(x_k - x_1) \cdots (x_k - x_{k-1})(x_k - x_{k+1}) \cdots (x_k - x_n)} = \frac{\prod_{j \neq k}(x - x_j)}{\prod_{j \neq k}(x_k - x_j)}$$

These L_k are known as a Lagrange basis. then it follows

$$p(x) = y_1 L_1(x) + \cdots + y_n L_n(x) = \sum_{i=1}^{n} y_i L_i(x)$$

Question: suppose $y_i = f(x_i)$. We consider the interpolation error $f(x) - p(x)$.

7 Lecture 13 - 08/10/26

Remark 7.1. We have $L_k(x_k) = 1,\ \forall k = 1, 2, \ldots, n$ and $L_k(x_j) = 0,\ \forall j \neq k$

Theorem 7.2. *Assume $p(x)$ is the Lagrange interpolating polynomial that interpolates $(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)$. Define $f(x_i) = y_i$. Then the interpolation error is*

$$f(x) - p(x) = \frac{(x - x_1)(x - x_2)\cdots(x - x_n)}{n!} f^{(n)}(\xi)$$

where ξ lies between the smallest and largest of $x_1, x_2, \ldots, x_n$.

Proof. If $x = x_i,\ i = 1, 2, \ldots, n$, then we are done since it vanishes. If $x \neq x_i$, define for any t between the smallest/largest of x_i

$$G(t) = [f(t) - p(t)] - \frac{(t - x_1)(t - x_2)\ldots(t - x_n)}{(x - x_1)(x - x_2)\ldots(x - x_n)}[f(x) - p(x)]$$

notice that $G(x_i) = 0$ and $G(x) = 0$. By MVT, $G(t)$ has $n + 1$ distinct zeroes which implies $G'(t)$ has n distinct zeroes, which cascades to $G^{(n)}(t)$ has a single unique zero ξ. Thus,

$$G^{(n)}(t) = f^{(n)}(t) - p^{(n)}(t) - \frac{n!}{\prod_{i=1}^{n} x - x_i}[f(x) - p(x)].$$

Notice the $p^{(n)}(t)$ term vanishes because L has degree $n - 1$. Then we can compute

$$0 = G^{(n)}(\xi) = f^{(n)}(\xi) - \frac{n!}{\prod_{i=1}^{n} x - x_i}[f(x) - p(x)].$$

□

Example 7.3. $y_i = \sin(x_i)$, $x_1 = 0, x_2 = \frac{\pi}{6}, x_3 = \frac{\pi}{3}, x_4 = \frac{\pi}{2}$.

$$\sin(x) - p(x) = \frac{(x - x_1)(x - x_2)(x - x_3)(x - x_4)}{4!} f^{(4)}(\xi)$$

Example 7.4. Interpolate

$$f(x) = \frac{1}{1 + 12x^2}$$

at evenly spaced points in $[-1, 1]$. When computing, the error is large at the endpoints. This is known as the Runge phenomenon.

7.1 Chebyshev nodes and polynomials

We want a way to reduce this interpolation error and Runge phenomenon.

Theorem 7.5. *The choice of points $x_1, x_2, \ldots, x_n$ in $[-1, 1]$ that makes the values of the numerator of the interpolation error*

$$\max_{-1 \leq x \leq 1} |(x - x_1)\cdots(x - x_n)|$$

as small as possible is

$$x_i = \cos\left[\frac{(2i - 1)\pi}{2n}\right], \quad i = 1, 2, \ldots, n$$

They are called the Chebyshev nodes and the minimum value is $\frac{1}{2^{n-1}}$. The minimum is achieved by

$$(x-x_1)(x-x_2)\cdots(x-x_n)=\frac{1}{2^{n-1}}T_n(x)$$

where $T_n(x)$ denotes the degree n Chebyshev polynomial.

Remark 7.6. The Chebyshev nodes distribute in a way such that there are less points in the middle and more points near the endpoints of the interval.

Example 7.7. $f(x)=e^x$ with 5 points. $p_4(x)$ is the degree 4 Chebyshev interpolating polynomial.

$$f(x)-p_4(x)=\frac{(x-x_1)\dots(x-x_5)}{5!}f^{(5)}(\xi)$$

then

$$\begin{aligned}x_1&=\cos\frac{\pi}{10}\\x_2&=\cos\frac{3\pi}{10}\\x_3&=\cos\frac{5\pi}{10}\\x_4&=\cos\frac{7\pi}{10}\\x_5&=\cos\frac{9\pi}{10}\end{aligned}$$

We also know that

$$|(x-x_1)\cdots(x-x_n)|\le\frac{1}{2^{n-1}}=\frac{1}{2^4}$$
$$|f^{(5)}(\xi)|=|e^{\xi}|\le e$$

$$|f(x)-p_4(x)|\le\frac{1}{2^4}\cdot\frac{1}{5!}e\approx 0.00142$$

Definition 7.8. The nth degree Chebyshev polynomial is given by

$$T_n(x)=\cos(n\arccos(x)),\ -1\le x\le 1$$

Remark 7.9. We have the following recursive relation:

$$\begin{cases}T_0(x)=1\\T_1(x)=x\\T_{n+1}(x)=2xT_n(x)-T_{n-1}(x)\end{cases}$$

Proof.

$$T_{n+1}=\cos[(n+1)y]=\cos(ny+y)=\cos(ny)\cos(y)-\sin(ny)\sin(y)$$
$$T_{n-1}=\cos(ny)\cos y+\sin(ny)\sin(y)$$
$$T_{n+1}+T_{n-1}=2\cos(ny)\cos(y)=2xT_n(x)$$

□

Remark 7.10. The following facts are true:

1. $T_n(x)$ is a polynomial.
2. $\deg T_n = n$ with leading coefficient 2^{n-1} is monic.
3. $T_n(1) = 1$ and $T_n(-1) = (-1)^n$.
4. $\max_{-1\le x\le 1} |T_n(x)| \le 1$.
5. Roots of $T_n(x) = 0$ are

$$\begin{aligned} \cos(n \arccos(x)) &= 0 \\ n \arccos x &= \frac{\pi}{2}(2i-1) \\ x_i &= \cos \frac{\pi}{2n}(2i-1),\ i = 1, 2, \dots, n \end{aligned}$$

   where these x_i are the Chebyshev nodes.
6. $T_n(x)$ alternates between -1 and 1 a total of $n+1$ times.

8 Lecture 14 - 13/10/26

Proof of Theorem 7.5. We prove by way of contradiction. Suppose we have the polynomial

$$p(x) = (x - \hat{x}_1)(x - \hat{x}_2) \cdots (x - \hat{x}_n)$$

and

$$\max_{-1\le x\le 1} |p(x)| < \frac{1}{2^{n-1}}$$

We want to construct

$$Q(x) = p(x) - \frac{1}{2^{n-1}} T_n(x)$$

Since $p(x)$ is bounded above and below by $\pm\frac{1}{2^{n-1}}$ and $T_n(x)$ oscillates between -1 and 1 by Fact 3, $Q(x)$ alternates between negative and positive values $n+1$ times, thus $Q(x)$ has n distinct roots. But since $p(x)$ is monic with degree $n-1$ and $T_n(x)$ has degree $n-1$, $Q(x)$ has at most degree $n-1$. Thus, we have a contradiction. Therefore,

$$\max_{-1\le x\le 1} |(x - \hat{x}_1)(x - \hat{x}_2) \cdots (x - \hat{x}_n)| \ge \frac{1}{2^{n-1}}$$

for any $\hat{x}$.
If $x_i = \cos(\frac{2i-1}{2n}\pi)$, then

$$\max_{-1\le x\le 1} |(x - \hat{x}_1)(x - \hat{x}_2) \cdots (x - \hat{x}_n)| = \frac{1}{2^{n-1}} T_n(x)$$

and

$$\max_{-1\le x\le 1} |(x - \hat{x}_1)(x - \hat{x}_2) \cdots (x - \hat{x}_n)| = \max_{-1\le x\le 1} |\frac{1}{2^{n-1}} T_n(x)| = \frac{1}{2^{n-1}}$$

□

8.1 Cubic spline interpolation

Definition 8.1. Given n data points $(x_1, y_1), \dots (x_n, y_n)$ with strictly increasing x_i, a cublic spline $S(x)$ through the data points is a set of cubic polynomials:

$$S_1(x) = y_1 + b_1(x - x_1) + c_1(x - x_1)^2 + d_1(x - x_1)^3 \text{ on } [x_1, x_2]$$
$$\vdots$$
$$S_i(x) = y_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3 \text{ on } [x_i, x_{i+1}]$$

for $i = 1, 2, \dots, n-1$.

Proposition 8.2. *The following properties hold for $S_i(x)$:*

1. $S_i(x_i) = y_i$ *and* $S_i(x_{i+1}) = y_{i+1}$
2. $S'_{i-1}(x_i) = S'_i(x_i)$
3. $S''_{i-1}(x_i) = S''_i(x_i)$
4. *The natural spline has the additional property of* $S''_1(x_1) = 0$ *and* $S''_{n-1} = 0$ *called end conditions.*

Proof. Property 1 direct computation:

$$y_{i+1} = y_i + b_i(x_{i+1} - x_i) + c_i(x_{i+1} - x_i)^2 + d_i(x_{i+1} - x_i)^3$$

The Property 2 and 3 computation is similar.
Property 4:

$$2c_1 = 0$$
$$2c_{n-1} + 6d_{n-1}(x_n - x_{n-1}) = 0 = 2c_n$$

Let $\delta_i = x_{i+1} - x_i$ and $\Delta_i = y_{i+1} - y_i$. this gives us a linear system

$$\begin{pmatrix} 1 & 0 & 0 & & \\ \delta_1 & 2\delta_1 + 2\delta_2 & \delta_2 & & \\ & \delta_2 & \ddots & \ddots & \ddots \\ & & \delta_{n-2} & 2\delta_{n-2} + 2\delta_{n-1} & \delta_{n-1} \\ & & & 0 & 1 \end{pmatrix} \begin{pmatrix} c_1 \\ c_2 \\ \vdots \\ c_n \end{pmatrix} = \begin{pmatrix} 0 \\ 3(\frac{\Delta_2}{\delta_2} - \frac{\Delta_1}{\delta_1}) \\ \vdots \\ 3(\frac{\Delta_{n-1}}{\delta_{n-1}} - \frac{\Delta_{n-2}}{\delta_{n-2}}) \\ 0 \end{pmatrix}$$

□

Example 8.3. Fine the natural spline through $(0, 3), (1, -2), and (2, 1)$. We have $x_1 = 0, x_2 = 1, x_3 = 2, y_1 = 3, y_2 = -2, y_3 = 1$. Then $\delta_1 = 1, \delta_2 = 1, \Delta_1 = -5, \Delta_2 = 3$. Using the tridiagonal system above, we get

$$\begin{pmatrix} 1 & 0 & 0 \\ 1 & 4 & 1 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} c_1 \\ c_2 \\ c_3 \end{pmatrix} = \begin{pmatrix} 0 \\ 24 \\ 0 \end{pmatrix}$$

which yields

$$\begin{cases} c_1 = 0 \\ c_2 = 6 \\ c_3 = 0 \end{cases}$$

Then we solve for d_i and b_i using the values solved for c_i. Then you end with

$$\begin{cases} S_1(x) = 3 - 7x + 2x^3 \text{ on } [0, 1] \\ S_2(x) = -2 - 1(x-1) + 6(x-1)^2 - 2(x-1)^3 \text{ on } [1, 2] \end{cases}$$

9 Lecture 15 - 15/10/26

9.1 Least squares method

The least squares method is a fitting model that attempts to minimize the residuals between the actual and predicted values.

Theorem 9.1.

$$A^T A\tilde{x} = A^T b, \quad A \in \mathbb{R}^{m\times n}$$

if and only if

$$Ax \perp b - A\tilde{x}$$

Definition 9.2. The root mean square error is

$$RMSE = \sqrt{\frac{r^T r}{m}}$$

Example 9.3. Find the least squares solution for

$$\begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 2 \\ 2 \\ 3 \\ 4 \end{bmatrix}$$

$$A^T A = \begin{bmatrix} 3 & 3 & 2 \\ 3 & 6 & 3 \\ 2 & 3 & 3 \end{bmatrix}$$

$$A^T b = \begin{bmatrix} 9 \\ 10 \\ 9 \end{bmatrix}$$

Then perform Gaussian elimination to with back substitution and get

$$\begin{cases} \tilde{x_1} = 2 \\ \tilde{x_2} = -\frac{1}{3} \\ \tilde{x_3} = 2 \end{cases}$$

The least-squares fitting model is as follows:

1. Given m data points, choose a model ($y = c_1 + c_2 t$)
2. Force the model to fit the data $Ac = b$, but this equation is impossible to solve because there are more equations than variables.
3. Solve the normal equation $A^T A\tilde{c} = A^T b$.

Remark 9.4. We have the following models:

1. Polynomial model: $y = c_1 + c_2 t + c_3 t^2 + \cdots + c_{n+1} t^n$
2. Periodic model: $y = a_0 + \sum_{n=1}^{N} a_n \cos(2n\pi t) + \sum_{n=1}^{N} b_n \sin(2n\pi t)$
3. Exponential model: $y = c_1 e^{c_2 t} \to_{\text{linearize}} \log(y) = \log(c_1) + c_2 t = k + c_2 t$. Then solve for $k, c_2 \implies c_1 = e^k, c_2 = c_2$.

4. Power law model: $y = c_1 t^{c_2} \to_{\text{linearize}} \log(y) = \log(c_1) + c_2 \log(t) = k + c_2 \log t$, then we have $c_1 = e^k, c_2 = c_2$.

Example 9.5. $(-1,1),(0,0),(1,0),(2,-2), \quad y = c_1 + c_2 t + c_3 t^2$.

$$Ac = b : \begin{pmatrix} 1 & -1 & 1 \\ 1 & 0 & 0 \\ 1 & 1 & 1 \\ 1 & 2 & 4 \end{pmatrix} \begin{pmatrix} c_1 \\ c_2 \\ c_3 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \\ 0 \\ -2 \end{pmatrix}$$

and after solve, we get

$$y = 0.45 - 0.65t - 0.25t^2$$

9.2 Conditioning of least-squares

Suppose we have $x_1 = 2, x_2 = 2.2, x_3 = 2.4, \ldots, x_{11} = 4.0$ which are equally spaced points. Then suppose we have the function

$$y_i = 1 + x_1 + x_2^2 + \cdots + x_i^7$$

Then we use the normal equation to fit the data using

$$p(x) = c_1 + c_2 x + \cdots + c_8 x^7$$

The matrix equation for the linear system is then

$$\begin{bmatrix} 1 & x_1 & x_1^2 & \cdots & x_1^7 \\ \vdots & & & & \\ 1 & x_{11} & x_{11}^2 & \cdots & x_{11}^7 \end{bmatrix} \begin{bmatrix} c_1 \\ c_2 \\ \vdots \\ y_{11} \end{bmatrix} = \begin{bmatrix} y_1 \\ \vdots \\ y_{11} \end{bmatrix}$$

Solving yields

$$\tilde{c} = \begin{bmatrix} 1.5134 \\ -0.2644 \\ \vdots \end{bmatrix}$$

But this is very far from the solution which is supposed to be 1. So then we get a bad least squares solution because our polynomial model did not fit the data well. So we want to find the least-squares solution using QR factorization. Then we get c very close to 1 and thus is a better conditioning.

9.3 QR factorization and Gram-Schmidt orthogonalization

Definition 9.6. The vector norm is defined as $\|u\| = \sqrt{u^T u}$

Definition 9.7. Two vectors $u, v \in \mathbb{R}^n$ are orthogonal if $u^T v = 0$

Definition 9.8. $\{q_1, q_2, \ldots, q_n\}$ are orthonormal if

$$q_i^T q_j = \begin{cases} 1 & \text{if } i = j \\ 0 & \text{else} \end{cases}$$

If $Q \in \mathbb{R}^{m \times n}, m \geq n$ has orthonormal column vectors,

$$Q^T Q = \begin{bmatrix} q_1^T & q_1^T q_2, \cdots, q_1^T q_n \\ \vdots & \ddots \end{bmatrix} = \mathrm{Id}_{n \times n}$$

Then with $Qx = b$, we use the normal equation to easily solve the system. To do this for general matrices, we use Gram-Schmidt to get $A = QR$.

Example 9.9. Find the least-squares to

$$\begin{bmatrix} \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{3}} \\ 0 & \frac{1}{\sqrt{3}} \\ \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{3}} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$$

We know

$$A^T A = \begin{bmatrix} a_1^T a_1 & a_1^T a_2 \\ a_2^T a_1 & a_2^T a_2 \end{bmatrix}$$

And since A has orthogonal columns,

$$A^T A = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$A^T b = \begin{bmatrix} \frac{1}{\sqrt{2}} \\ -\frac{1}{\sqrt{3}} \end{bmatrix}$$

Thus

$$\tilde{c} = \begin{bmatrix} \frac{1}{\sqrt{2}} \\ -\frac{1}{\sqrt{3}} \end{bmatrix}$$

Example 9.10. $AQ = R$, Q has orthonormal columns, $m \geq n$, R is upper triangular.

$$\begin{aligned} Ax &= b \\ QRx &= b \\ Q^T QRx &= Q^T b \\ Rx &= Q^T b \end{aligned}$$

We also have

$$\begin{aligned} Ax &= b \\ A^T A\tilde{x} &= A^T b \end{aligned}$$

10 Lecture 16 - 20/10/26

10.1 Review of interpolation methods

1. Lagrange interpolation: for normal cases
2. Least-squares method: If we have data points with the same x, we have more equations than unknowns. We may also even have an inconsistent system, so we have no classical solution. For this case we use the normal equation and the least-squares method.
3. Better LSM: use QR factorization to solve the normal equation better using Gram-Schmidt.

Example 10.1. Make the columns orthogonal:

$$A = \begin{bmatrix} 1 & -1 & 4 \\ 1 & 4 & -2 \\ 1 & 4 & 4 \\ 1 & 1 & 2 \\ a_1 & a_2 & a_3 \end{bmatrix}$$

Note a_i is not actually in the matrix, just label for columns

1. $$r_{11} = \sqrt{a_1^T a_1} = \sqrt{4} = 2$$
$$q_1 = \frac{a_1}{r_{11}} = \begin{bmatrix} 1/2 \\ 1/2 \\ 1/2 \\ 1/2 \end{bmatrix}$$
2. make y_2 orthogonal to q_1 then normalize
$$y_2 = a_2 - r_{12}q_1, \quad r_{12} = q_q^T a_2$$
$$r_{22} = \|y_2\| = 5, \quad q_2 = \frac{y_2}{r_{22}}$$
3. make y_3 orthogonal to q_1 and q_2 then normalize
$$r_{13} = q_t^T a_3, \quad r_{23} = q_2^T a_3$$
$$y_3 = a_3 - r_{13}1_1 - r_{23}q_2, \quad r_{33} = \|y_3\| = 4, \quad q_3 = \frac{y_3}{r_{33}}$$

Therefore, we have

$$Q = \begin{bmatrix} q_1 & q_2 & q_3 \end{bmatrix} = \begin{bmatrix} 1/2 & -1/2 & 1/2 \\ 1/2 & 1/2 & -1/2 \\ 1/2 & 1/2 & 1/2 \\ 1/2 & -1/2 & -1/2 \end{bmatrix}$$

,

$$R = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ 0 & r_{22} & r_{23} \\ 0 & 0 & r_{33} \end{bmatrix}$$

and one can check that $A = QR$.

10.2 Gram-Schmidt orthogonalization for least-squares method

For $A_1, A_2, \ldots, A_n$ linearly independent vectors in $\mathbb{R}^m$, $n \leq m$,

1. $y_1 = A_1, \quad q_1 = \frac{y_1}{\|y_1\|}$
2. $y_2 - A_2 - q_1(q_1^T A_2), \quad q_2 = \frac{y_2}{\|y_2\|}$
   $\vdots$
3. $y_j = A_j - q_1(q_1^T A_j) - \cdots - q_{j-1}(q_{j-1}A_j), \quad q_j = \frac{y_j}{\|y_j\|}$

Then we also can find $r_{ij} = q_1^{\perp} A_j$, so

$$(A_1|A_2|\cdots|A_n) = (q_1|\cdots|q_n)\begin{pmatrix} r_{11} & r_{12} & \cdots & r_{12} \\ 0 & r_{22} & \cdots & r_{2n} \\ & & \ddots & \vdots \\ & & & r_{nn} \end{pmatrix}$$

We can then solve the least-squares equation with our QR factorization. Suppose A is full rank, i.e. $A \in R^{m\times n}, (m \geq n)$. We also have Q has orthonormal columns, and R is an upper triangular matrix. Then

$$\begin{aligned} A\tilde{x} &= b \\ QR\tilde{x} &= b \\ R\tilde{x} &= Q^T b \end{aligned}$$

1. Use Gram-Schmidt to get QR.
2. compute $c = Q^T b$
3. do back substitution to solve $R\tilde{x} = c$

10.3 Classical Gram-Schmidt algorithm

Let $A_j, j = 1, 2, \ldots, n$ be linearly Independent

```
for j = 1,2,..., n
  y = A_j
  for i = 1,2,...,j-1
      r_ij = q_i^T*A_j
      y = y-r_ij q_i
  end
  r_jj = norm(y) and q_j = y / r_jj
end
```

10.4 Special case with small numbers

Example 10.2. Perform Gram-Schmidt method to solve least-squares

$$A = \begin{bmatrix} 1 & 1 & 1 \\ \delta & 0 & 0 \\ 0 & \delta & 0 \\ 0 & 0 & \delta \end{bmatrix}, \quad \delta = 10^{-10}$$

$$y_1 = A_1 = \begin{bmatrix} 1 \\ \delta \\ 0 \\ 0 \end{bmatrix}, \quad q_1 = \frac{1}{\sqrt{1+\delta^2}} \begin{bmatrix} 1 \\ \delta \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ \delta \\ 0 \\ 0 \end{bmatrix}$$

$$y_2 = \begin{bmatrix} 1 \\ 0 \\ \delta \\ 0 \end{bmatrix} - \begin{bmatrix} 1 \\ \delta \\ 0 \\ 0 \end{bmatrix} q_1^T A_2 = \begin{bmatrix} 0 \\ -\delta \\ \delta \\ 0 \end{bmatrix}, \quad q_2 = \begin{bmatrix} 0 \\ -\frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \\ 0 \end{bmatrix}$$

$$q_3 = \begin{bmatrix} 0 \\ -\frac{1}{\sqrt{2}} \\ 0 \\ \frac{1}{\sqrt{2}} \end{bmatrix}$$

But notice $q_2^T q_3 = \frac{1}{2} \neq 0$ so they are not orthonormal. This is due to our very small δ. But if we do the Modified Gram-Schmidt (below), we get

$$y_3' = \begin{bmatrix} 1 \\ 0 \\ 0 \\ \delta \end{bmatrix} - \begin{bmatrix} 1 \\ \delta \\ 0 \\ 0 \end{bmatrix} q_1^T A_3 = \begin{bmatrix} 0 \\ -\delta \\ 0 \\ \delta \end{bmatrix}, \quad y_3 = y_3' - \begin{bmatrix} 0 \\ -\frac{1}{\sqrt{2}} \\ 1\sqrt{2} \\ 0 \end{bmatrix} y_3' \implies q_3 = \begin{bmatrix} 0 \\ -\frac{1}{\sqrt{6}} \\ -\frac{1}{\sqrt{6}} \\ \frac{2}{\sqrt{6}} \end{bmatrix}$$

and thus we end up with

$$q_2^T q_3 = 0$$

10.5 Modified Gram-Schmidt algorithm

Let $A_j, j = 1, 2, \ldots, n$ be linearly Independent

```
for j = 1,2,..., n
  y = A_j
  for i = 1,2,...,j-1
      r_ij = q_i^T*y
      y = y-r_ij q_i
  end
  r_jj = norm(y) and q_j = y / r_jj
end
```

10.6 Reduced QR

Example 10.3. Say we have $Am \times n, Qm \times n, Rn \times n$.

$$A = \begin{bmatrix} 1 & -4 \\ 2 & 3 \\ 2 & 2 \end{bmatrix} = QR$$

$$Q = \begin{bmatrix} 1/3 & -14/15 \\ 2/3 & 1/3 \\ 2/3 & 2/15 \end{bmatrix}, \quad R = \begin{bmatrix} 3 & 2 \\ 0 & 5 \end{bmatrix}$$

For full QR, take $A_3 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$. So

$$y_3 = A_3 - q_1(q_1^T A_3) - q_2(q_2^T A_3) \implies q_3 = \frac{y_3}{\|y_3\|}$$